Table des matières

1. Introduction
2. Nuage de points
3. Tracés linéaires
4. Tracés linéaires simples
5. Regression

Introduction

La visualisation de données est la présentation de données avec des graphiques. C'est un moyen de résumer vos conclusions et de les afficher sous une forme qui facilite l'interprétation et peut aider à identifier des modèles ou des tendances. Avoir de superbes visualisations de données rendra votre travail plus intéressant et plus clair. Dans ce cahier, vous apprendrez à créer des nuages de points et des graphiques linéaires.

Nuage de points

Un nuage de points utilise des points et des coordonnées cartésiennes pour afficher la position des valeurs de deux ou plusieurs variables. Il est possible de tracer des nuages de points en 2D et 3D.

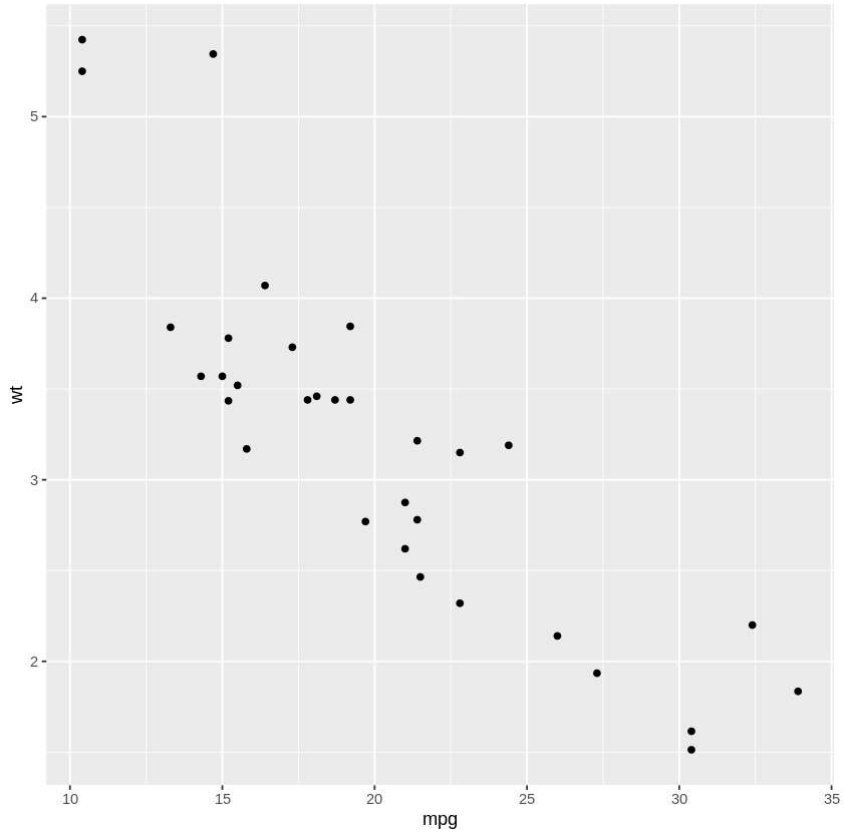
Commençons par importer la bibliothèque `ggplot2` .

```
if("ggplot2" %in% rownames(installed.packages()) == FALSE) {install.packages("ggplot2")}
library(ggplot2)
```

Nous pouvons créer un nuage de points 2D très simple en utilisant simplement qplot. L'utilisation de deux variables comme paramètres crée un nuage de points par défaut. Ici, nous utilisons le jeu de données mtcars.

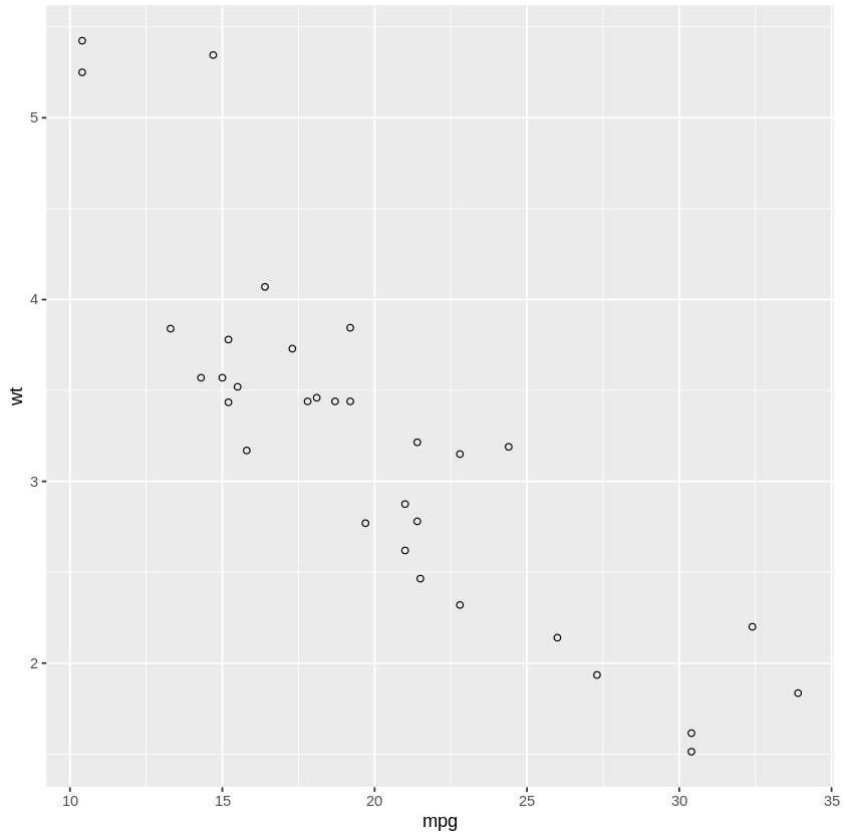
```
qplot(mpg, wt, data=mtcars)
```

```
Warning message:
“`qplot()` was deprecated in ggplot2 3.4.0.”
```



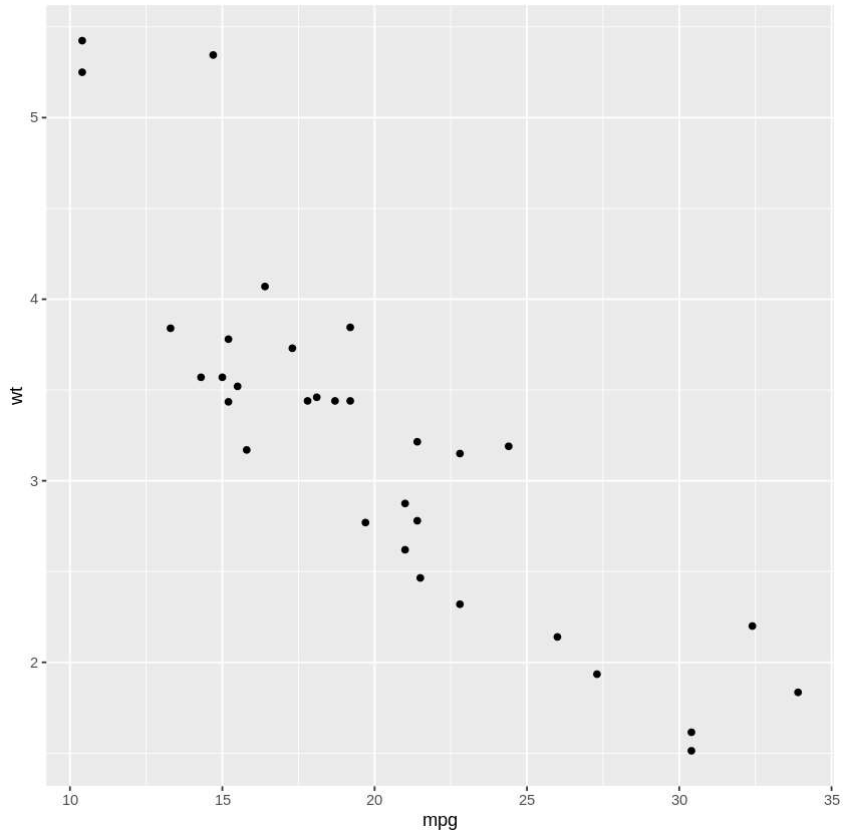
Nous pouvons créer un graphique similaire en utilisant ggplot.

```
ggplot(mtcars, aes(x = mpg, y = wt)) + geom_point(shape=1)
```



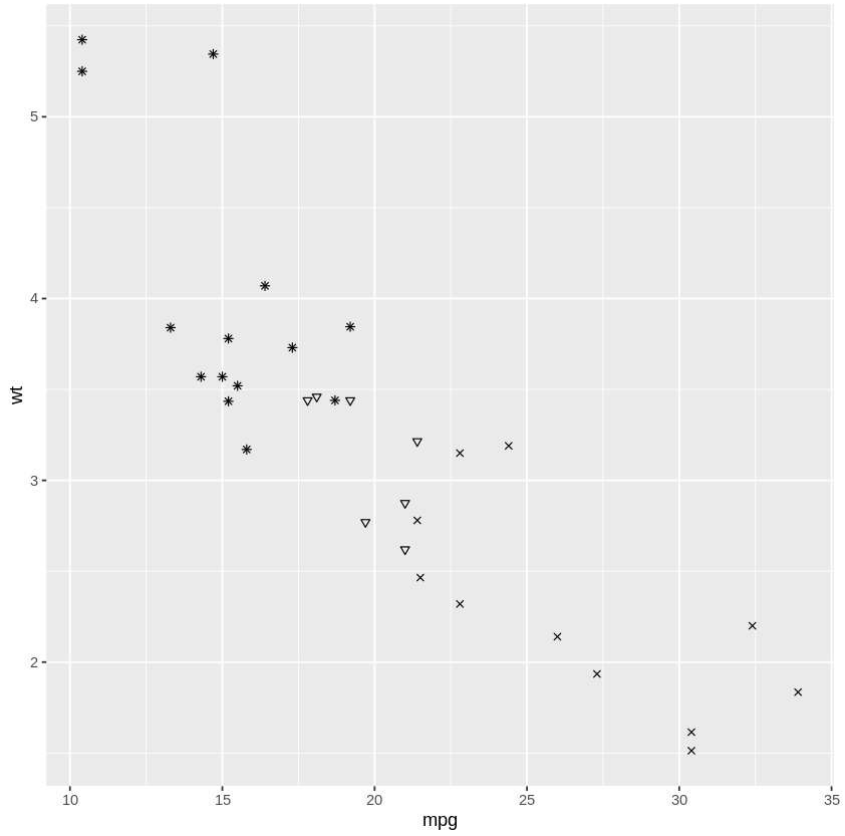
Vous avez dû remarquer que la forme des cercles a changé. On peut avoir des cercles pleins modifiant le paramètre `geom_point`.

```
ggplot(mtcars,aes(x=mpg,y=wt)) + geom_point(shape=19)
```



Comme nous pouvons contrôler les formes, nous pouvons en tirer parti et tracer trois variables dans un nuage de points bidimensionnel. Il vous suffit de passer le paramètre 'shape' en tant que colonne présente dans votre dataset.

```
ggplot(mtcars,aes(x=mpg,y=wt,shape = cyl)) + geom_point() + scale_shape_identity()
```

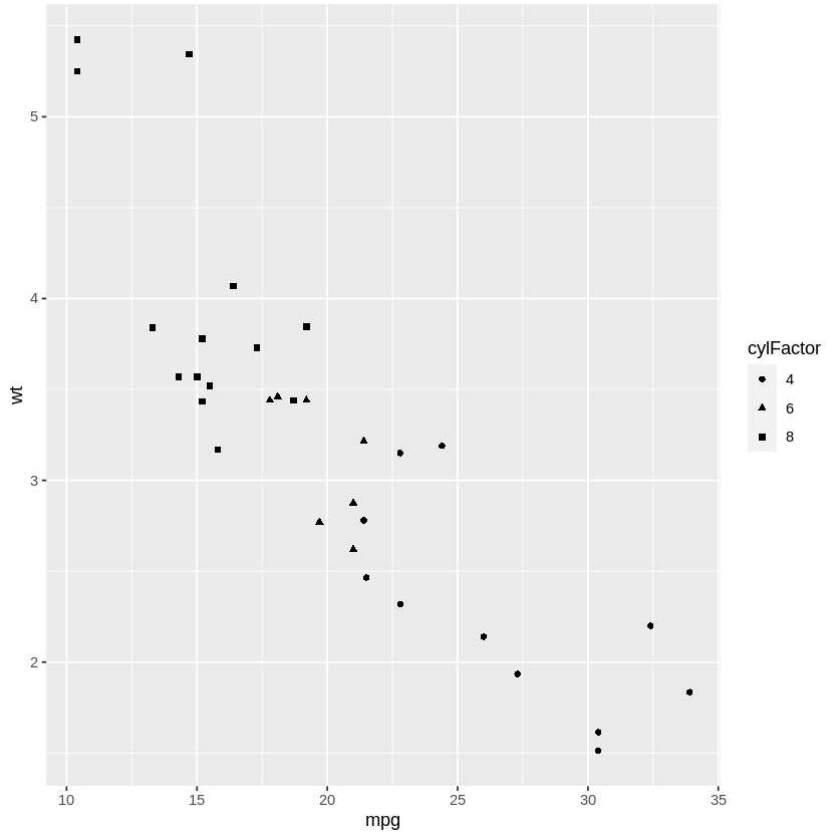


Cela n'a pas fonctionné. C'est parce que nous avons besoin d'avoir des données catégorielles, au lieu de numériques. Nous pouvons corriger ce facteur de création pour les cylindres.

```
mtcars$cylFactor<- factor(mtcars$cyl)
```

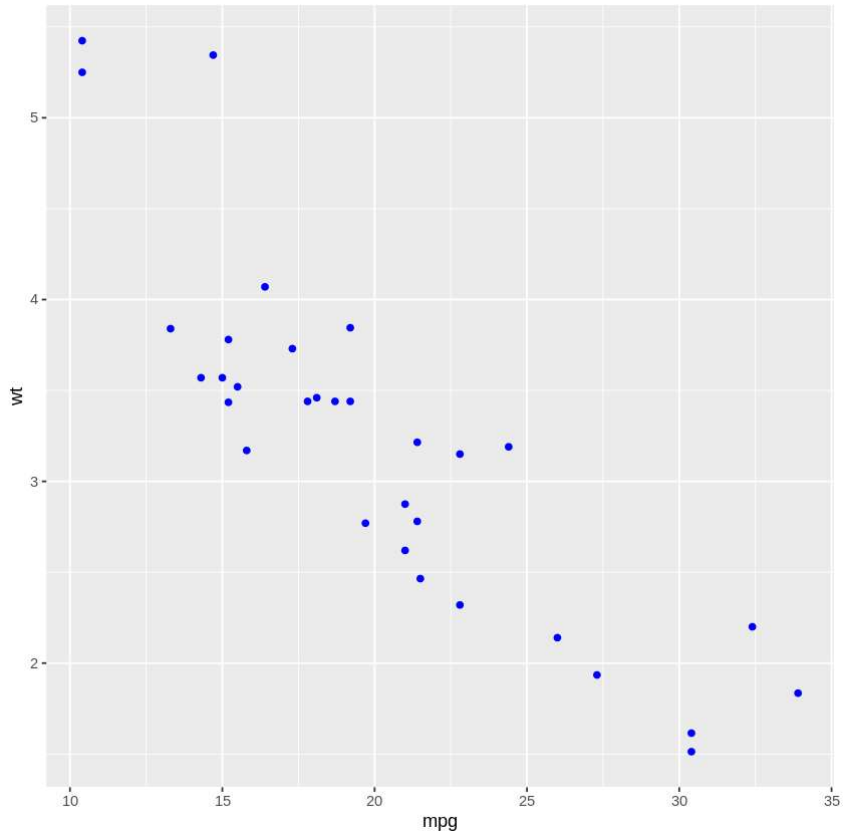
Il ne nous reste plus qu'à tracer à nouveau notre graphique.

```
ggplot(mtcars,aes(x=mpg,y=wt,shape = cylFactor)) + geom_point()
```



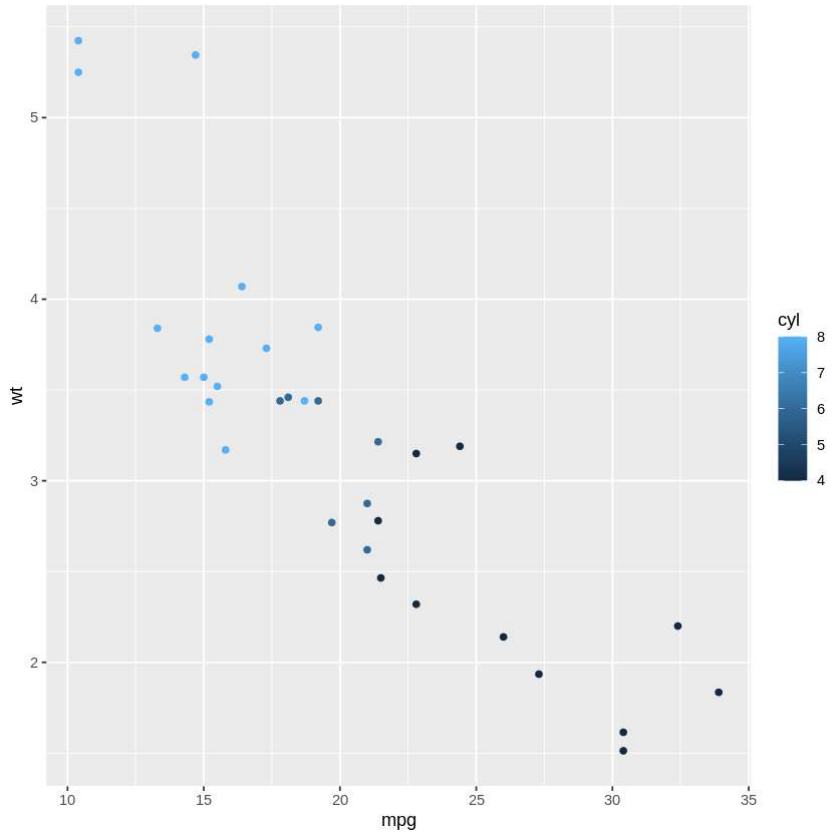
Nous pouvons changer les couleurs de nos cercles en utilisant le paramètre 'couleur'

```
ggplot(mtcars,aes(x=mpg,y=wt)) + geom_point(shape=19, colour="blue")
```



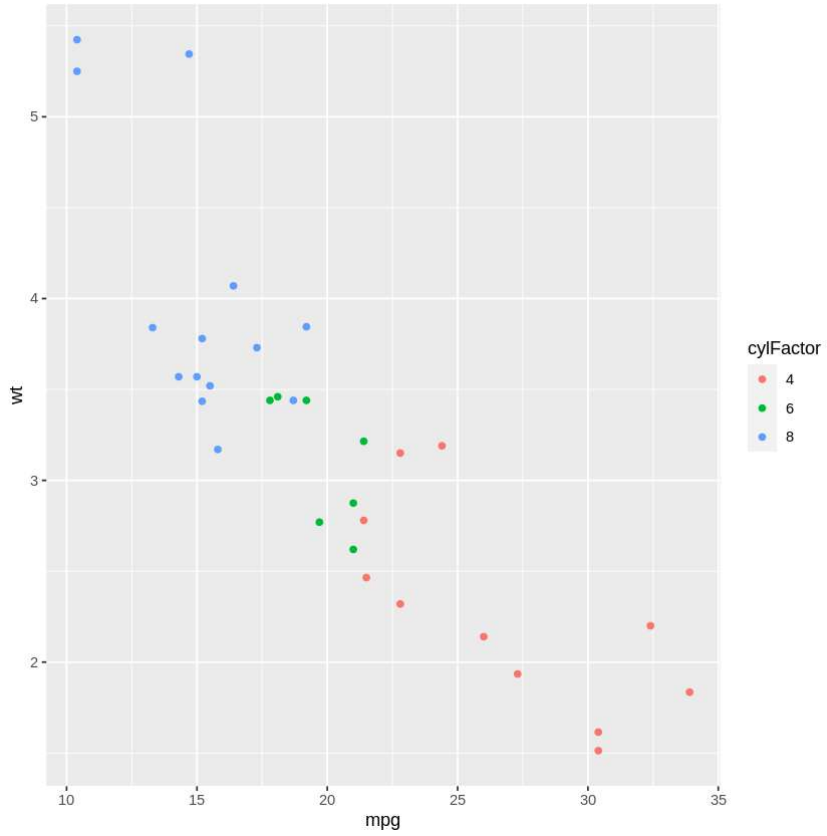
Comme nous l'avons fait avec les formes, nous pouvons faire la même chose avec les couleurs. Encore une fois, vous passez simplement le paramètre `couleur` en tant que colonne présente dans votre ensemble de données.

```
ggplot(mtcars,aes(x=mpg,y=wt,color = cyl))+ geom_point(shape=19)
```



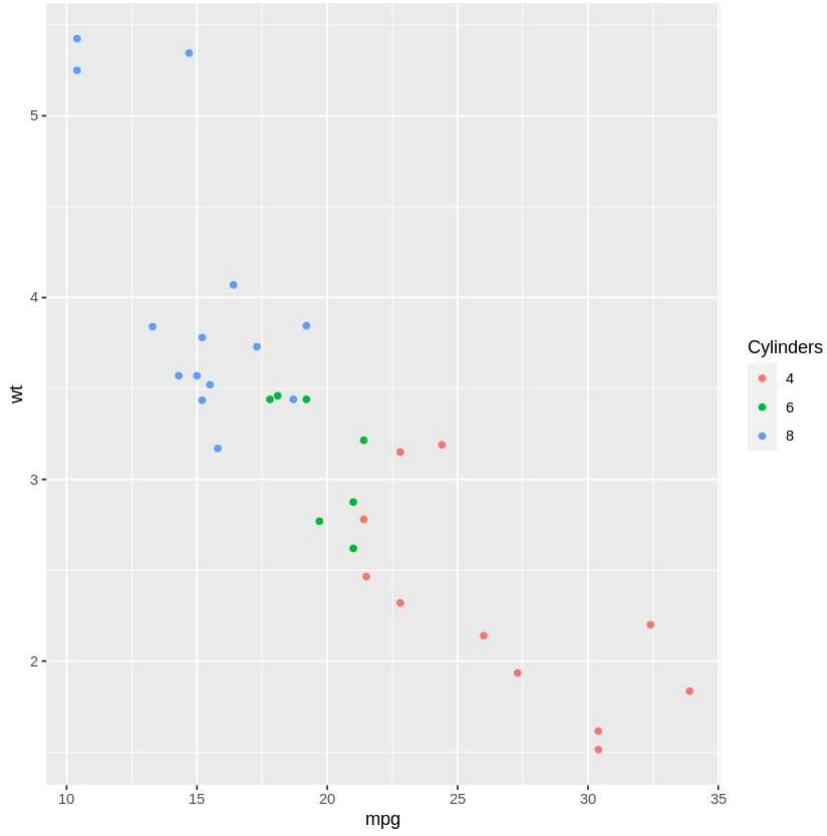
Comme le graphique précédent utilisait des données numériques, il créait l'étiquette avec un gradient de 4 à 8. Si nous utilisons des données catégorielles, comme les « cylFactors », nous pouvons attribuer une couleur unique à chaque catégorie.

```
ggplot(mtcars,aes(x=mpg,y=wt,color = cylFactor)) + geom_point(shape=19)
```



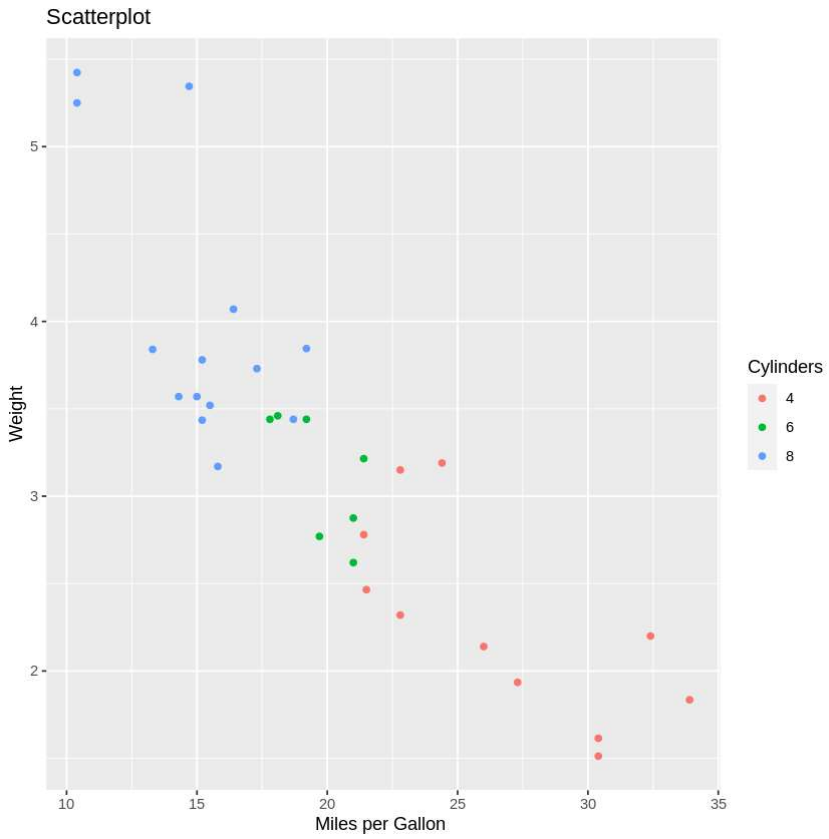
Maintenant, nous devons renommer notre légende. Nos utilisateurs ne comprendraient pas ce qu'est exactement cylFactors.

```
ggplot(mtcars,aes(x=mpg,y=wt,color = cylFactor)) + geom_point(shape=19) + labs(colour = "
```



Juste pour terminer notre graphique, nous pouvons renommer et créer les étiquettes restantes.

```
ggplot(mtcars,aes(x=mpg,y=wt,color = cylFactor)) + geom_point(shape=19) + xlab("Miles per
  labs(colour = "Cylinders") + ggtitle("Scatterplot")
```



Tracés linéaires

Les tracés linéaires sont également des représentations de données dans lesquelles les coordonnées cartésiennes sont utilisées. Tout comme les nuages de points, les données sont transformées en points - cependant, dans les graphiques linéaires, elles sont reliées par des lignes, comme leur nom l'indique.

Tracés linéaires simple

Pour les tracés linéaires simples, nous allons utiliser le jeu de données EuStockMarkets. Il s'agit également d'un ensemble de données inclus avec R et qui décrit les données historiques de quatre marchés boursiers européens.

Commençons à regarder leur fichier d'aide

```
?EuStockMarkets
```

Comme nous pouvons le voir, il contient 1860 observations pour chacun, et leur type de fichier est `mts`, qui signifie matrice. `ggplot2` ne fonctionne pas avec les matrices.

Pour contourner cette limitation, nous devons créer une base de données à partir de cette matrice. Heureusement, il existe une fonction qui fait exactement ce dont nous avons besoin. C'est le `as.data.frame()`.

```
EuStockDF <- as.data.frame(EuStockMarkets)
```

Maintenant, regardons les premières lignes de cette base de données, pour voir comment les données sont structurées.

```
head(EuStockDF)
```

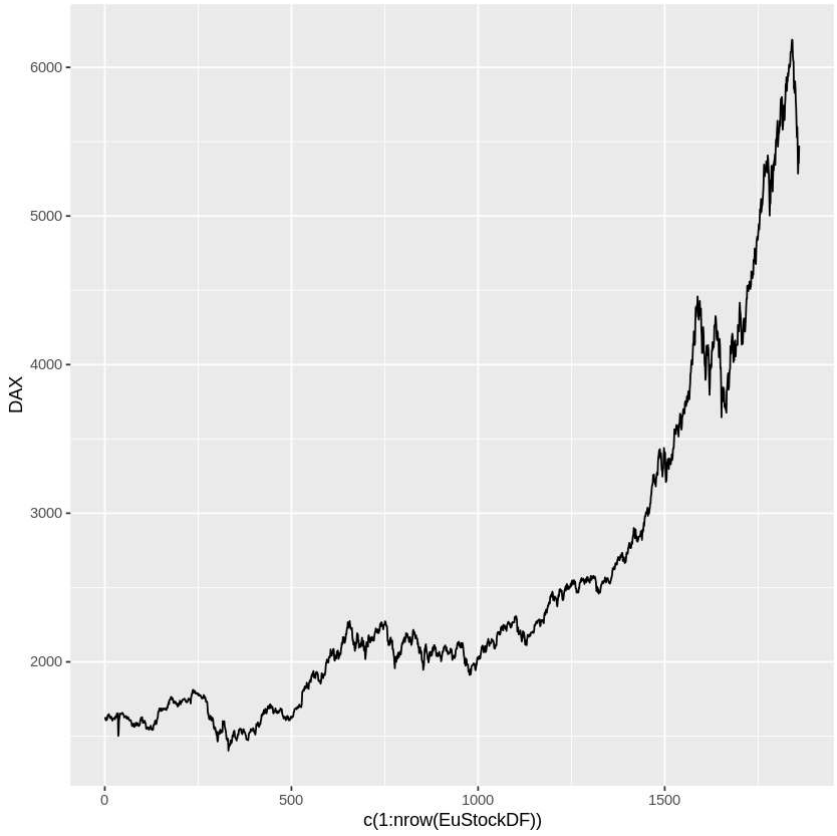
A data.frame: 6 × 4

	DAX	SMI	CAC	FTSE
	<dbl>	<dbl>	<dbl>	<dbl>
1	1628.75	1678.1	1772.8	2443.6
2	1613.63	1688.5	1750.5	2460.2
3	1606.51	1678.6	1718.0	2448.2
4	1621.04	1684.1	1708.1	2470.4
5	1618.16	1686.6	1723.1	2484.7
6	1610.61	1671.6	1714.3	2466.8

Maintenant, commençons à tracer !

Créons un graphique linéaire du cours de l'action "DAX":

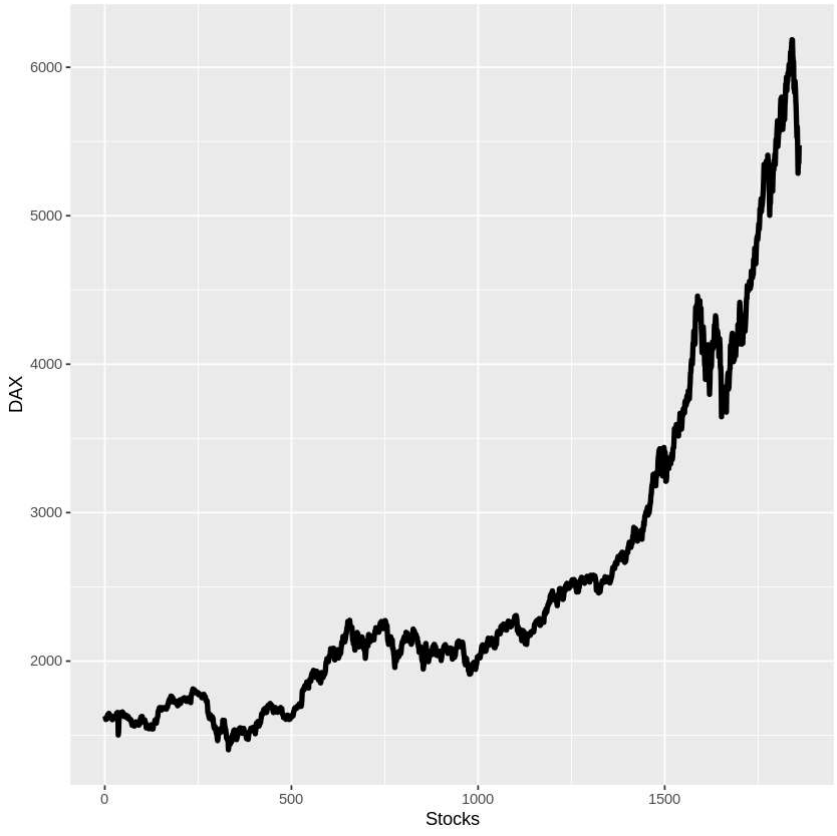
```
ggplot(EuStockDF,aes(x=c(1:nrow(EuStockDF)), y = DAX)) + geom_line()
```



Augmentons la largeur de notre ligne :

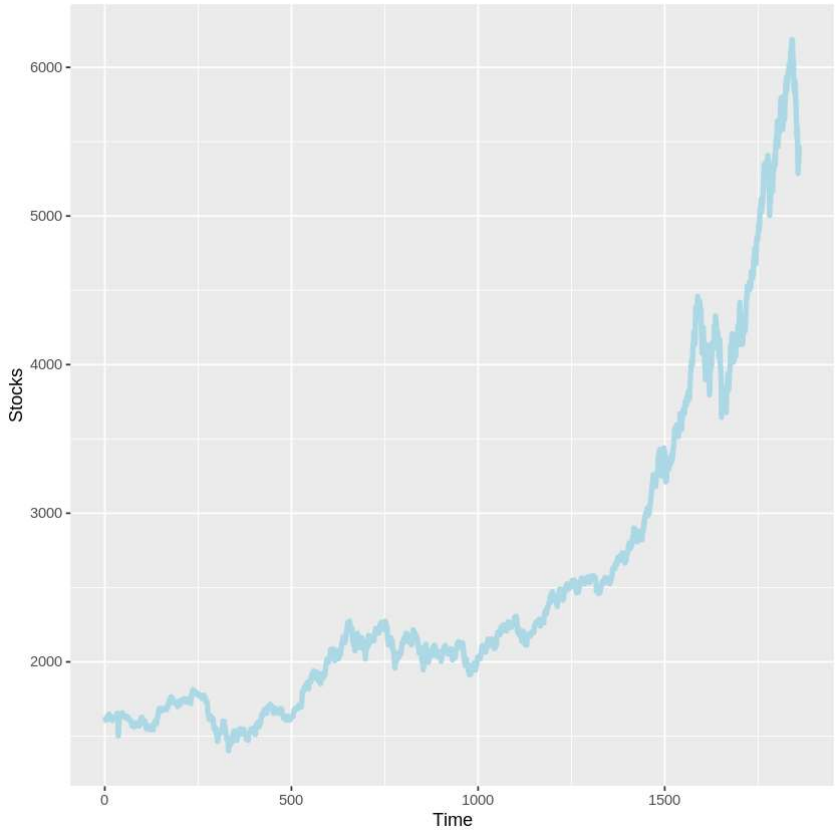
```
ggplot(EuStockDF,aes(x=c(1:nrow(EuStockDF)), y = DAX)) + geom_line(size=1.5) + labs(x = "
```

```
Warning message:
“Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.”
```



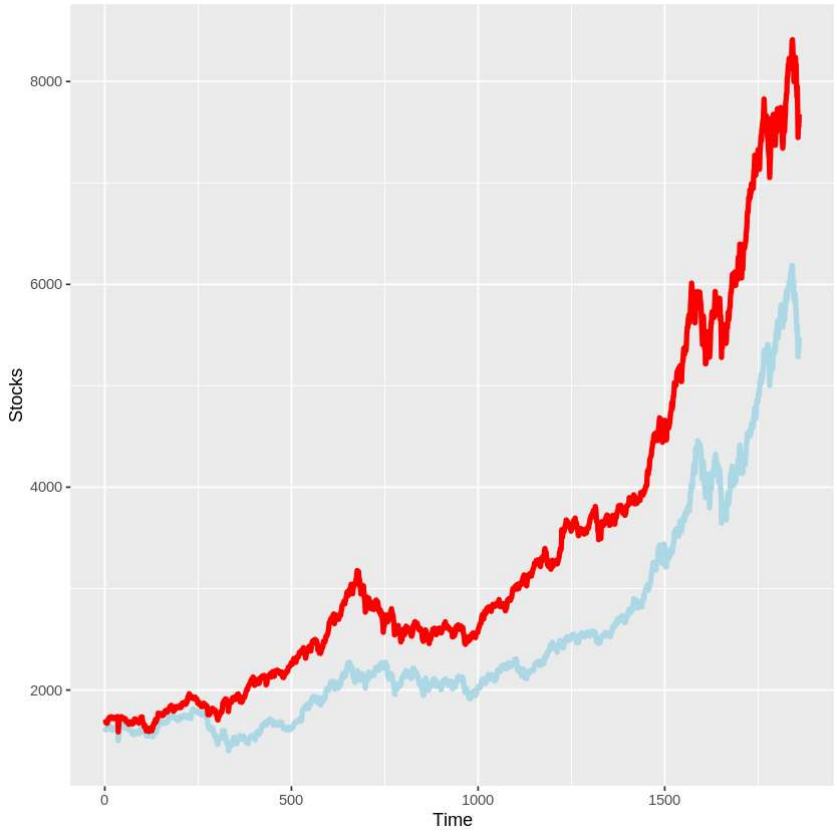
Pour changer la couleur, nous pouvons utiliser le paramètre `couleur`.

```
ggplot(EuStockDF,aes(x=c(1:nrow(EuStockDF)), y = DAX)) + geom_line(size=1.5, colour="ligh
```



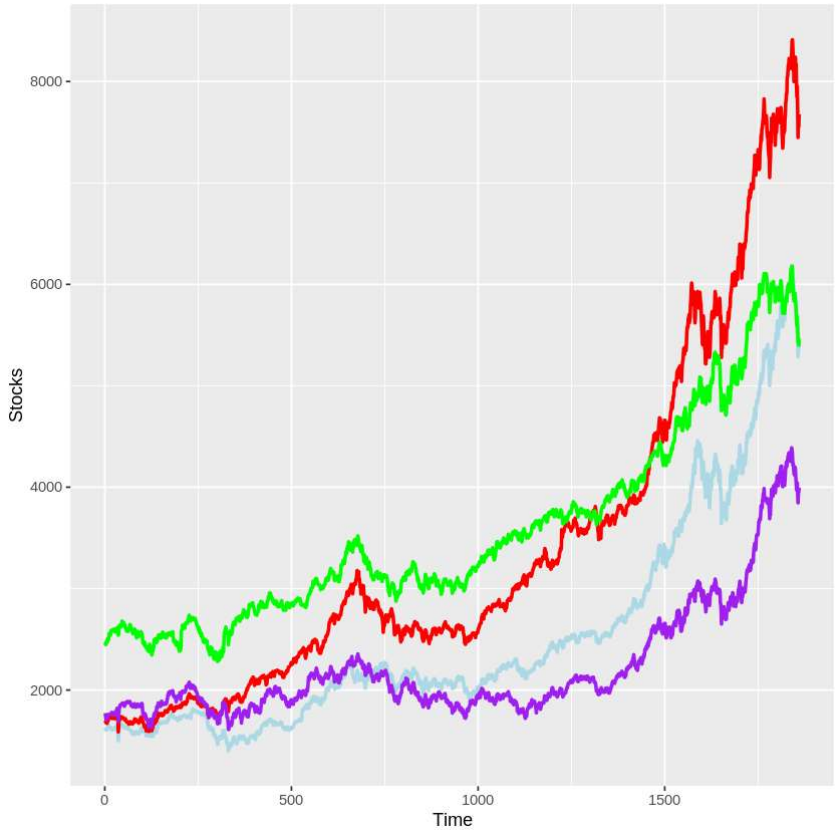
Nous pouvons également tracer plus d'une ligne à la fois dans le même graphique. Essayons de tracer deux actions dans le même graphique.

```
dax_smi_plot <- ggplot() +
  geom_line(data = EuStockDF,aes(x=c(1:nrow(EuStockDF)), y = DAX), size = 1.5, colour="li
  geom_line(data = EuStockDF,aes(x=c(1:nrow(EuStockDF)), y = SMI), size = 1.5, colour = "
  labs(x = "Time", y = "Stocks")
print(dax_smi_plot)
```



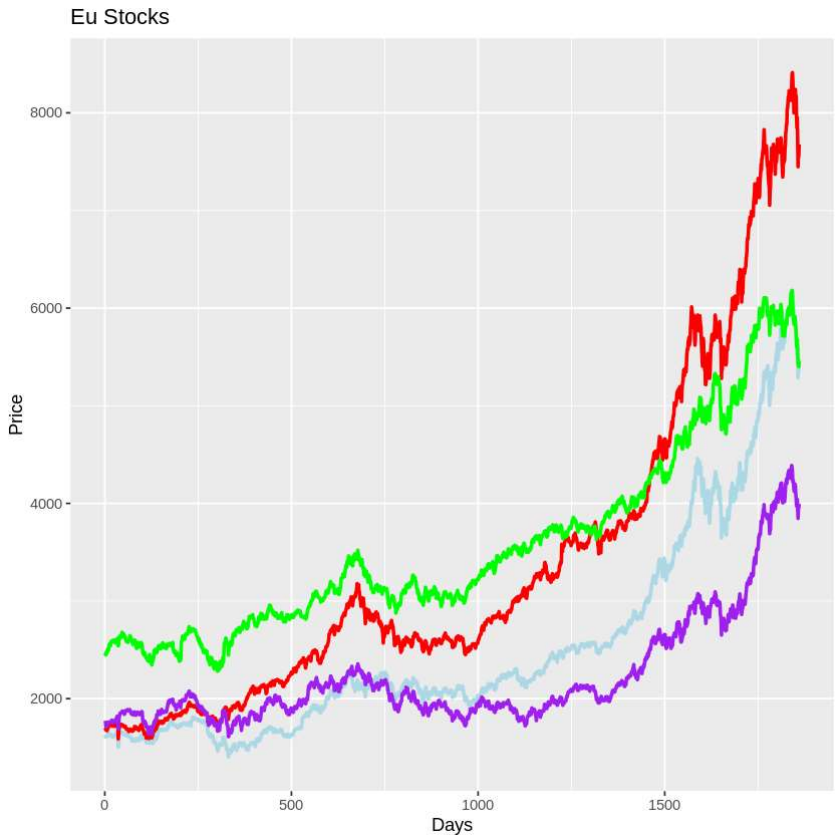
Maintenant, traçons tous les stocks.

```
all_stocks <- ggplot() +
  geom_line(data = EuStockDF,aes(x=c(1:nrow(EuStockDF)), y = DAX), size=1, colour="light
  geom_line(data = EuStockDF,aes(x=c(1:nrow(EuStockDF)), y = SMI), size =1, colour = "red
  geom_line(data = EuStockDF,aes(x=c(1:nrow(EuStockDF)), y = CAC), size =1, colour = "pur
  geom_line(data = EuStockDF,aes(x=c(1:nrow(EuStockDF)), y = FTSE), size =1, colour = "gr
  labs(x = "Time", y = "Stocks")
print(all_stocks)
```



Modifions les étiquettes dans notre graphique.

```
legend_stocks <- all_stocks + xlab("Days") + ylab("Price") + ggtitle("Eu Stocks")
print(legend_stocks)
```



Regression

La régression est un concept de modélisation mathématique dans lequel une formule linéaire est créée pour simuler le comportement perçu des points de données pour lesquels vous créez le modèle. La régression est un excellent moyen de visualiser les tendances et de créer des prédictions de base.

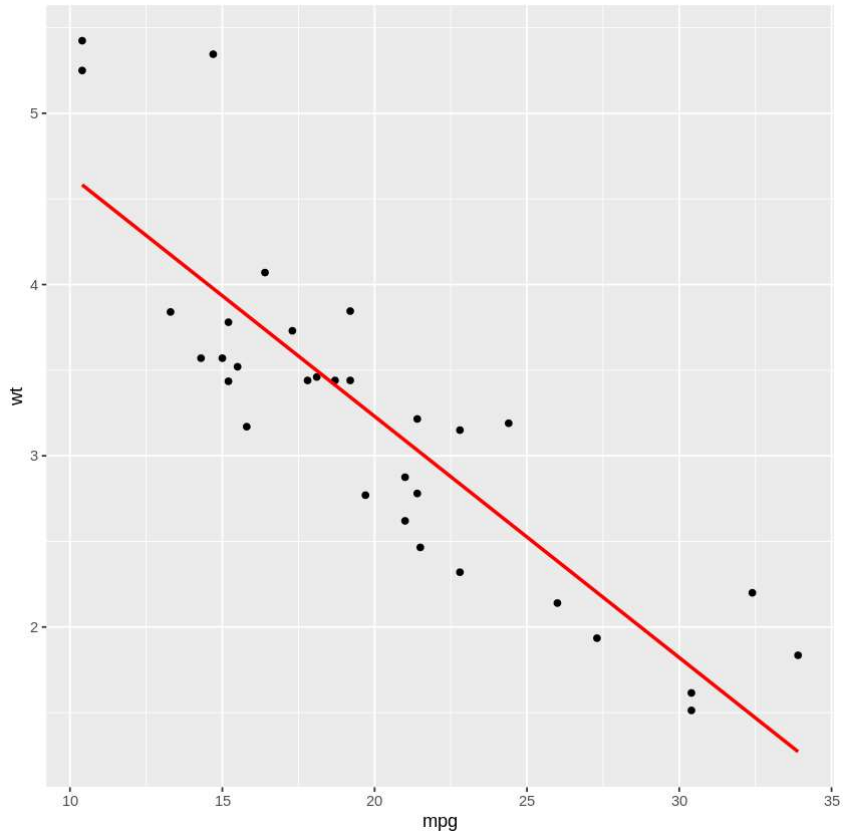
Les prédictions sont faites sur un ensemble de variables, étant donné un autre ensemble de variables. Les tendances sont généralement extraites de la relation entre deux ensembles de variables. Dans cet exemple, nous allons vérifier s'il existe une relation entre le poids et les miles par gallon d'une voiture.

Pour visualiser la régression, nous pouvons tracer notre ligne d'ajustement sur nos graphiques. Utilisons le même exemple de la section précédente et dessinons notre droite de régression par-dessus.

La première méthode que nous utiliserons est la régression linéaire, qui est une régression qui utilise ce que nous appelons un modèle linéaire. Pour ce faire, nous passons le paramètre `lm` à l'attribut `method` de la fonction `geom_smooth`, comme ceci :

```
ggplot(mtcars,aes(x=mpg,y=wt))  + geom_point(shape=19) +
  geom_smooth(method="lm", se= FALSE, color = "red")
```

```
`geom_smooth()` using formula = 'y ~ x'
```



Vous avez peut-être remarqué que notre code a `se = FALSE` dessus. `se` représente ce que nous appelons "l'intervalle de confiance" de notre modèle. Les intervalles de confiance sont des limites inférieures et supérieures dans lesquelles nous pouvons avoir une certitude à 95 % qu'un point de données se trouvera.

Si vous souhaitez afficher cet intervalle, vous pouvez soit ignorer l'attribut `se`, soit le définir comme `TRUE`.

```
#se = TRUE  -> confidence interval appear (default = true)
ggplot(mtcars,aes(x=mpg,y=wt,color = cylFactor)) + geom_point(shape=19) + geom_smooth(met
```

```
`geom_smooth()` using formula = 'y ~ x'
```

